DAY 9

Embeddings & Vector Search

Theory Notes for the 4-Week Agentic AI Learning Journey

CORE IDEA

Convert text into numerical vectors, store those vectors, and retrieve passages by semantic similarity rather than relying only on exact keyword matches.

LEARNING STACK

Python | Ollama | nomic-embed-text | PostgreSQL | pgvector

A practical theory guide covering embeddings, vector similarity, vector databases, pgvector, indexing, retrieval, limitations, and the Day 8 → Day 9 pipeline.

1. Why do we need vector search?

Traditional databases are excellent at storing structured information and retrieving it using filters, exact values, and text conditions. Traditional keyword search looks for terms that appear in a document. This can fail when a user expresses an idea using different words from the source.

Example: A document says “vehicles offload computationally intensive tasks to roadside edge servers.” A user asks “Where do cars send demanding computations?” The wording differs, but the meaning is closely related.

Semantic search addresses this by representing text as vectors. Texts with related meanings are expected to have nearby representations in the embedding space.

Keyword search vs semantic search

Keyword search	Semantic / vector search
Matches words or lexical patterns.	Matches learned semantic representations.
Can miss synonyms or paraphrases.	Can retrieve related wording without exact term overlap.
Useful for exact identifiers, names, and phrases.	Useful for natural-language questions and conceptual retrieval.
Often uses inverted indexes / text search.	Uses embeddings and vector similarity.

2. What is an embedding?

An embedding is a numerical representation of an input—such as a word, sentence, paragraph, image, or other data object—in a vector space. For text retrieval, an embedding model maps a piece of text to a list of floating-point numbers.

```
[0.12, -0.08, 0.34, 0.91, ...]
```

The numbers are not manually assigned labels such as “security = 0.8.” They are learned representations produced by a model. Their useful property is that geometric relationships in the vector space can reflect patterns of semantic relatedness.

How text becomes an embedding

- **Tokenization:** Text is split into tokens that the model can process.
- **Token representations:** The model maps tokens into internal numerical representations.
- **Contextual processing:** Transformer layers process relationships among tokens and their context.
- **Pooling or projection:** Token-level information is combined or transformed into a fixed-size representation for the whole input.
- **Output vector:** The embedding model returns a vector with a defined number of dimensions.

The exact internal architecture and pooling strategy depend on the embedding model. The important practical point is that the same model should be used consistently for document chunks and search queries.

3. Dimensions and embedding models

Vector dimensions are the number of numeric components in an embedding. In this project, nomic-embed-text produces 768-dimensional embeddings. Each chunk is therefore represented by 768 values.

If a table stores vectors with 768 dimensions, each inserted embedding must have exactly 768 values. A different dimension is incompatible with that column definition.

Why dimensions matter

- Higher dimensionality can provide more representational capacity, but it also increases storage and computation costs.
- Lower dimensionality can reduce storage and speed up operations, but may lose some representational detail.
- Dimension count alone does not determine quality. The model, training, data, and task all matter.

Model consistency

Document embeddings and query embeddings must come from compatible embedding spaces. If documents were embedded using one model and queries using another unrelated model, their vector distances may not be meaningful. Changing the embedding model generally requires regenerating and re-indexing the stored document vectors.

In this project, both document chunks and user queries are embedded with nomic-embed-text through Ollama.

4. Understanding vector similarity

After converting a query and each document chunk into vectors, the system needs a measure of how close those vectors are. Common measures include cosine similarity, dot product, and Euclidean distance.

Cosine similarity

Cosine similarity measures the cosine of the angle between two vectors. It focuses on their direction rather than their absolute magnitude.

For vectors A and B:

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)
```

A value closer to 1 indicates a smaller angle and greater directional similarity. A value around 0 indicates orthogonality; a negative value indicates an angle greater than 90 degrees. The exact score distribution depends on the embedding model and data.

Cosine distance in pgvector

pgvector's <=> operator calculates cosine distance. In the Day 9 search query, similarity is derived as:

```
similarity = 1 - cosine_distance
```

The SQL expression used by the project is:

```
1 - (embedding <=> query_vector) AS similarity
```

A similarity score is not a probability or a percentage of correctness. For example, 0.76 does not mean that the result is 76% correct. It is a distance-derived ranking signal.

Other distance measures

- **Dot product:** Multiplies corresponding vector components and sums the results. Its scale can depend on vector magnitudes.
- **Euclidean distance (L2):** Measures straight-line distance between two points. Smaller distance means closer vectors.

The best metric depends on the embedding model's intended use and how its vectors are normalized or trained.

5. What is a vector database?

A vector database stores vectors and supports efficient similarity search over them. It commonly stores metadata alongside vectors so retrieved results can be connected to their original documents, pages, users, or records.

A vector database is useful when an application needs to retrieve items based on similarity rather than exact equality. Examples include semantic document search, recommendation, image similarity, and retrieval for RAG.

Do we need a separate vector database?

Not always. Existing databases can gain vector-search capabilities through extensions or built-in features. This project uses PostgreSQL with pgvector, so relational data and embeddings can live in the same database.

PostgreSQL + pgvector

pgvector is a PostgreSQL extension, not a separate database. It adds a vector data type, distance operators, and indexing methods to PostgreSQL.

- PostgreSQL continues to manage tables, transactions, SQL, constraints, and relational data.
- pgvector adds vector storage and nearest-neighbor search.
- The application can join retrieved chunks with ordinary metadata such as source filename and page number.

6. Exact search and approximate nearest neighbors

A nearest-neighbor search finds the vectors closest to a query vector.

Exact nearest-neighbor search

An exact search evaluates the distance against all eligible vectors and returns the true nearest results for that set. It is conceptually straightforward and can be appropriate for smaller collections, but the work grows as the collection becomes larger.

Approximate nearest-neighbor (ANN) search

ANN methods use an index to find very good nearby candidates without exhaustively comparing the query with every vector. This can greatly reduce query time, but the returned neighbors may not always be the exact mathematical top results.

Common pgvector index types

Index	High-level idea	Trade-off
HNSW	Builds a multi-layer graph connecting nearby vectors.	Often offers strong query performance and recall; can use more memory.
IVFFlat	Partitions vectors into lists and searches selected lists.	Can be faster and smaller to build; quality and speed depend on tuning.

The Day 9 project focuses on getting the retrieval pipeline working. It does not need an ANN index to demonstrate the basic vector-search concept; a database can perform exact distance ordering for a modest dataset.

7. Top-K retrieval

Top-K retrieval means returning the K closest chunks to the query according to the selected distance metric. If K = 5, the application returns five candidate chunks.

A simplified retrieval flow:

1. Receive the user's question.
2. Generate an embedding for the question.
3. Compare it with stored chunk embeddings.
4. Order by cosine distance (ascending).
5. Return the first K rows.

The value of K is a design choice. A small K can omit useful context. A very large K can add irrelevant material, increase prompt size, and consume more LLM context. K should be evaluated on representative questions.

Metadata matters

The vector alone is not enough for a useful document application. Metadata such as source filename, page number, chunk index, and document identifier lets the application display provenance and trace an answer back to its source.

8. The Day 8 → Day 9 pipeline

Day 8 produced document chunks using different chunking strategies: fixed-size, recursive, and semantic. Day 9 consumes a selected chunk JSON file, generates an embedding for each chunk, and stores the text and vector in PostgreSQL.

Stage	Responsibility
Document loading	Extract text from source files such as PDFs or TXT files.
Chunking (Day 8)	Split long documents into smaller pieces suitable for retrieval.
Embedding (Day 9)	Convert each chunk into a 768-dimensional vector using nomic-embed-text.
Storage	Save chunk content, embedding, and metadata in PostgreSQL.
Query embedding	Convert the user's question using the same embedding model.
Similarity search	Rank stored vectors by cosine distance and return Top-K chunks.

9. What the Day 9 implementation does

Embedding generation

The embedder sends text to Ollama's embedding API using the configured model. It validates that the expected number of embeddings is returned and that each vector has the expected 768 dimensions.

Database schema

The project's `document_chunks` table stores a unique chunk key, document identifier, source, page number, chunk index, text content, vector embedding, metadata, and creation timestamp.

```
embedding VECTOR(768)
```

The vector column's declared dimension ensures that stored vectors conform to the selected embedding model's output size.

Ingestion

The ingestion script reads the configured chunk JSON file, extracts chunk text and metadata, generates embeddings in batches, and inserts records. A stable chunk key and conflict handling help avoid inserting the same chunk more than once.

Search

The search script embeds the user's query and executes a SQL query ordered by cosine distance. It returns the chunk content, source, page number, and a similarity score.

Cleanup

The cleanup script deletes rows from the `document_chunks` table after confirmation. It leaves the table structure and pgvector extension intact. This is useful when switching between fixed, recursive, and semantic chunking outputs.

10. Understanding the retrieval results

In the project test, the database contained 1,807 chunks. A query about blockchain in vehicular edge computing returned several passages from the source paper. A broad query initially returned general VEC passages, while a more specific question about the consensus mechanism retrieved a passage describing Proof-of-QK Stake (PoQKS).

This illustrates both the strength and limitation of semantic retrieval: it can find relevant passages even when the query does not contain the exact technical term, but the highest-ranked chunks are not guaranteed to contain a complete answer.

Why can a high score still be irrelevant?

- A query can be semantically related to a broad topic while asking for a specific fact.
- A chunk may mention the topic without explaining the requested relationship.
- Chunk boundaries may separate a definition from its explanation.
- The embedding model's similarity ranking is not a fact-checking mechanism.

Evaluate retrieval using a set of realistic questions and inspect whether the returned passages actually support the answer. Do not judge the system using similarity scores alone.

11. Limitations and ways to improve retrieval

- **Chunking quality:** Poor chunk boundaries can split related information or create chunks that are too broad.
- **Embedding quality:** A general embedding model may not rank specialized domain content perfectly.
- **Top-K trade-off:** Too few results may omit evidence; too many may add noise.
- **Keyword-sensitive queries:** Exact terms, identifiers, formulas, and names may benefit from keyword search alongside vector search.
- **Metadata filtering:** Filtering by document, source, date, or category can narrow the search space.
- **Hybrid retrieval:** Combining lexical search with vector similarity can improve coverage for queries that mix concepts and exact terms.
- **Reranking:** A reranker can examine the query and candidate passages more directly and reorder candidates before passing them to an LLM.

These are possible improvements, not features that are necessarily implemented in the current Day 9 code.

12. Vector search vs RAG

Day 9 implements retrieval: it finds and returns relevant document chunks. A complete Retrieval-Augmented Generation (RAG) system adds an LLM that uses those retrieved chunks to formulate an answer.

> **Vector retrieval:**
> Question → Query embedding → Vector search → Top-K chunks
> **Basic RAG:**
> Question → Retrieve Top-K chunks → Build context → LLM prompt → Grounded response

The retrieved passages should be included in the LLM context, and the prompt should instruct the model to answer using that context and acknowledge when the context does not contain the answer. Retrieval quality remains important: an LLM cannot reliably use evidence that was not retrieved.

13. Interview revision questions

Q1. What is an embedding?

A. A learned numerical representation of data, such as text, in a vector space.

Q2. Why use vector search instead of keyword search?

A. Vector search can retrieve semantically related content even when exact words differ. Keyword search remains useful for exact terms and identifiers.

Q3. What does 768-dimensional mean?

A. Each embedding contains 768 numeric components.

Q4. What is cosine similarity?

A. A measure based on the cosine of the angle between two vectors, commonly used to compare embedding directions.

Q5. What does pgvector provide?

A. A PostgreSQL extension that adds vector storage, distance operators, and vector indexing methods.

Q6. Is pgvector a separate database?

A. No. It is an extension that adds vector capabilities to PostgreSQL.

Q7. What is Top-K retrieval?

A. Returning the K nearest candidate vectors for a query.

Q8. What is the difference between exact search and ANN?

A. Exact search finds the true nearest neighbors by evaluating the eligible vectors; ANN uses an index to search faster with possible recall trade-offs.

Q9. What is HNSW?

A. A graph-based approximate nearest-neighbor indexing method.

Q10. What is IVFFlat?

A. An index that clusters vectors into lists and searches selected lists.

Q11. Does a similarity score represent answer correctness?

A. No. It is a similarity signal, not a probability or correctness score.

Q12. Why must query and document embeddings be compatible?

A. They must inhabit compatible vector spaces for distance comparisons to be meaningful.

Q13. Is Day 9 a complete RAG system?

A. No. It implements the retrieval component. RAG adds an LLM that generates an answer using retrieved context.

14. Key takeaways

- Embeddings represent text as numerical vectors.
- Vector similarity enables semantic retrieval beyond exact keyword matching.
- The Day 9 project uses Ollama's nomic-embed-text model to produce 768-dimensional vectors.
- PostgreSQL plus pgvector stores and searches embeddings alongside document metadata.
- Cosine distance is used to rank candidate chunks; similarity scores are not correctness probabilities.
- Top-K retrieval returns candidates, not guaranteed answers.
- Day 9 provides the retrieval foundation for the next step: a RAG pipeline that connects retrieved context to a local LLM.

End of Day 9 theory notes. The notes describe the concepts and the implementation discussed in this learning project. Indexing, hybrid search, reranking, and full RAG are presented as related concepts or future improvements where applicable.